

Cover Letter

Forward Deployed Engineer - AI Engineer, Applied AI

Gichan Lee | +82 10-6666-2824 | gichanlee@icloud.com | Portfolio: <https://bit-habit.com> | GitHub: github.com/bookseal

1. I already do FDE work — intent to production

Reflection describes this role as owning "the technical strategy and delivery of agentic systems from initial customer discovery through production launch." That sentence is my current job.

As Lead AI FDE, I translate executives' business intent into deployed agents. The system I designed connects three roles: non-developer clients who state intent, AI agents that execute, and developers who verify — with the agent's work made clear enough for non-developers to drive, and developer judgment encoded as explicit policy. Agent actions are branched by type: reads run automatically, board writes need per-item human approval, and deletions, permission changes, and code changes are prohibited. That guardrail design is the difference between an agent demo and an agent you can put in front of a real customer.

2. Agentic systems, deployed across cloud and on-prem

The role asks for agentic systems on state-of-the-art models, deployed across hybrid environments — public cloud, VPC, on-premises. I've built both halves and connected them.

The agent half: BookToss is a LangGraph agent that searches 25 library systems with no public API, wrapping each site as a tool and handling retries in a state graph. It's not a demo — it's deployed on a cluster I run, observed by Prometheus, and has been live for two years. For structured extraction I use forced tool_use with Pydantic validation as a safety layer.

The deployment half: I run a k3s cluster of 14 services with zero downtime — ArgoCD GitOps, ingress/TLS automation, observability I keep public at status.bit-habit.com. I built on AWS, then re-architected and migrated the whole thing to OCI with no downtime, so I've felt the real problems of moving production between environments: ordering, dual-running, cutting dependencies, and catching the "it runs but not like before." I also built a k8s scale-to-zero launcher that brings LLM apps up 0→1 on demand and back to zero when idle.

3. Evaluations, not vibes

The JD lists evaluations alongside RAG and agent orchestration. This is where I do my most careful work.

I built a RAG system over 1,297 pages of aviation law and, instead of tuning by eyeballing a few questions, wrote an **eval harness that ran 45 configurations** (chunking × embedding × retrieval × K) overnight against a blind holdout, scored by coverage/recall/MRR — and verified that every cited section actually existed in the retrieved chunks. I picked the deployed config by a rule, "within noise, take the cheaper one," which cut tokens 38%, and I killed a better-scoring agentic loop because it cost too much per answer. That entry was graded 9/10, top of its class. For an FDE putting AI in front of enterprise customers, "how confident, and how do we know" is the whole job.

4. Why Reflection, and what I'm still learning

Reflection's mission — making intelligence open and accessible so anyone can control and build on it — is the version of this field I actually want to work in. And "a fast-paced startup where playbooks are still being written" describes how I already operate: I build end to end, alone, and turn what I learn into documentation so the next person moves faster. Everything I've made is running right now; I'd rather show you something that works than ask you to trust a claim.

I'll be honest about the edges. My deepest work is in RAG, evaluations, and agent orchestration; **fine-tuning is the area I've touched least** — bootcamp and side-project level, not production. I'm not going to overstate it. But adapting models for a customer's needs sits right next to the eval and inference work I do daily, and closing that gap is exactly the kind of learning I'd join to do.

Portfolio — <https://bit-habit.com> · live services, agent projects, and the RAG eval harness, each linked to its repo.